Oracle APEX in Action

Writing Your First SQL Query and Your First Script

Courses	CSCI 6950 - Advanced Database Design and Administration | CSCI 4850
Instructor	Angello Astorga | amastorga@ysu.edu
Teaching Assistant	Sundar Raj Sharma | ssharma33@student.ysu.edu
Term	Fall 2026
Platform	Oracle APEX Cloud - Object Browser, SQL Commands, SQL Scripts
Estimated time	45 to 60 minutes
Graded	No. This is a warm-up before the graded assignments.

Part 1 | About This Demo

Overview

This is a guided tour of Oracle APEX. You will use three tools in the SQL Workshop and see what each one is good for. You will create a table, put records into it, query those records, change them, and remove them. Then you will take the same work and turn it into a saved SQL script that runs on its own.

The point is the difference between the tools. Object Browser lets you inspect a database without writing SQL. SQL Commands runs one statement at a time and is ideal for testing an idea. SQL Scripts runs many statements together and saves them under a name, so the whole job repeats on demand. Real database work uses all three.

The demo is not graded. Plan on 45 to 60 minutes. Everything here uses a throwaway table named TEST_STUDENTS with made-up records, so nothing refers to a real person and you can rebuild the table as often as you like.

What You Will Learn

- What each of the three SQL Workshop tools does, and when to reach for it.
- How to create a table and choose a data type for each column.
- How to add, read, change, and remove records.
- How to filter, sort, and summarize the rows a query returns.
- How to write a SQL script, save it, run it, and read the run report.
- How to read a common Oracle error message and fix the statement.

Goals

- Give you a safe table to explore, so mistakes cost nothing.
- Make the basic SQL commands familiar before the graded work starts.
- Show why a saved script beats retyping the same statements by hand.
- Build the habit of testing a WHERE clause with SELECT before you run an UPDATE or a DELETE.

Learning Outcomes

After you finish this document, you will be able to:

1. Use the Object Browser to inspect a table without writing SQL.
2. Run single statements in SQL Commands and read the results grid.
3. Create a table with CREATE TABLE and pick a suitable data type for each column.
4. Add records with INSERT and retrieve them with SELECT.
5. Filter rows with WHERE, LIKE, IN, BETWEEN, and IS NULL, and sort them with ORDER BY.
6. Summarize data with COUNT, AVG, MIN, MAX, GROUP BY, and HAVING.
7. Change records with UPDATE, remove them with DELETE, and change a table with ALTER TABLE.
8. Write, save, and run a multi-statement SQL script, and check its run report.
9. Explain when a script is the better choice than a single command.
10. Identify common Oracle errors and correct the statement that caused them.

Concepts Covered

Concept	Category	Where it appears
Browsing a schema without SQL	Tooling	Part 2
Creating a table and choosing data types	DDL	Part 3
Adding records	DML	Part 4
Querying, filtering, and sorting	DML	Part 5
Aggregation and grouping	DML	Part 6
Changing and removing records	DML	Part 7
Changing a table structure	DDL	Part 7

Concept	Category	Where it appears
Saved multi-statement scripts	Tooling	Part 8
NULL values	Data integrity	Parts 4 and 5
Saving your work with COMMIT	Transactions	Parts 2 and 8

DDL means Data Definition Language, the commands that build and change the structure of a database. DML means Data Manipulation Language, the commands that work with the data inside it. Both terms come up again throughout the course.

How This Prepares You for Later Assignments

Every graded assignment assumes you can create a table, query it, and rebuild it without help. This demo gives you that base and sets up what comes next.

What comes later	What you build on here
Designing a schema with several related tables	Creating one table and choosing its columns
Primary keys, foreign keys, and constraints	Column data types, and how NULL behaves
Joining two or more tables in one query	Writing SELECT with WHERE and ORDER BY
Reports and summary queries	GROUP BY, HAVING, and the aggregate functions
Setting up an assignment schema in one step	Saved SQL scripts and the run report
Indexes and query performance	The results area and the Explain tab

The table you build here is small on purpose. The commands and the scripting habits are the same ones you will use on a much larger schema later.

How to Use This Document

Parts 2 through 8 are the walkthrough. Work through them in order, because each part uses the data created in the part before it. Part 9 holds a short practice set with solutions, and Part 10 lists common errors.

The layout separates the two kinds of content. Plain text explains what a command does and why it matters. Anything you type into APEX appears in a shaded box labeled SQL or SCRIPT. Read the text first, then run the command.

DOCUMENT MAP

Part 2 | The Three Tools

The SQL Workshop has five tools across the top. This demo uses three of them. Each one solves a different problem, and knowing which to reach for saves a lot of time.

Tool	What it does	Reach for it when
Object Browser	A point-and-click view of your schema, with no SQL required	You want to look at a table structure or its records
SQL Commands	Runs one statement and shows the result below it	You are testing a single query or a single change
SQL Scripts	Saves many statements under a name and runs them together	You want to repeat a job, or hand it to someone else

Object Browser

Click a table on the left and you get a set of tabs. The Columns tab shows the structure. The Data tab shows the records. Constraints and Indexes come later in the course.

Nothing in the Object Browser requires SQL. That makes it the fastest way to check whether a command you ran actually worked.

> **Schema selector:** The Schema dropdown in the top right controls which schema you are viewing. Every table you create goes into the schema selected there. If a table seems to have disappeared, check this setting first.

SQL Commands

SQL Commands is a box where you type a statement and press Run. Results appear directly below. You will use it for Parts 3 through 7.

> **One statement at a time:** SQL Commands runs a single statement per Run. Two statements pasted together produce an error. Clear the box between statements. This limit is exactly why SQL Scripts exists.

READING THE RESULTS AREA

Control	What it does
Rows dropdown	Sets how many rows appear, and defaults to 10. A query that returns more rows still shows only the first 10 until you raise this value. Your data is not missing.
Results tab	The rows your query returned. This is the default tab.
Explain tab	The plan Oracle uses to run your query. You will use this later in the course.
Describe tab	The structure of a table, without running a query.
Saved SQL tab	Statements you kept with the Save button.
History tab	Statements you ran recently. This is the fastest way to recover one you cleared by mistake.
Download link	Exports the current results to a file.

SQL Scripts

SQL Scripts holds a named file of SQL statements. You write the statements once, save the script, and run the whole thing whenever you need it. APEX then shows a run report listing every statement and whether it succeeded.

Part 8 covers this tool in full. For now, keep the difference in mind. SQL Commands is for one idea at a time. SQL Scripts is for a job you will run more than once.

> **If your rows vanish later:** SQL Commands normally commits your work automatically, so an INSERT is saved as soon as it runs. If you return in a new session and the records are gone, run COMMIT; once after your inserts and check again. In a script, always add COMMIT; yourself.

Part 3 | Create a Table

A table holds data in rows and columns. Each column needs a name and a data type. The statement below creates a table with six columns.

Open SQL Commands, type this, and click Run. It is one statement, so it runs on its own.

```
SQL
CREATE TABLE test_students (
  student_id   NUMBER,
  first_name   VARCHAR2(30),
  last_name    VARCHAR2(30),
  major        VARCHAR2(40),
  gpa          NUMBER(3,2),
  enrolled_on  DATE
);
```

You should see the message "Table created."

Six columns is more than the minimum, and that is on purpose. Text, numbers, and dates each behave differently in a query. You need all three to practice properly.

> **If you see ORA-00955:** A table named test_students already exists in your schema. Run DROP TABLE test_students; first, then create it again.

THE DATA TYPES YOU JUST USED

Type	Holds	Example
NUMBER	Any number, whole or decimal	student_id NUMBER
NUMBER(p,s)	A number with p digits in total and s digits after the decimal point	gpa NUMBER(3,2)
VARCHAR2(n)	Text up to n characters, stored at the length you use	first_name VARCHAR2(30)
CHAR(n)	Text of exactly n characters, padded with spaces	state_code CHAR(2)
DATE	A date and time down to the second	enrolled_on DATE
TIMESTAMP	A date and time with fractional seconds	created_at TIMESTAMP

- Use VARCHAR2 rather than CHAR for names, emails, and most text. CHAR pads short values with spaces, and those spaces cause confusing results when you compare values.
- The number in VARCHAR2(30) is a limit, not a reservation. A five character value uses five characters, not thirty.
- NUMBER(3,2) allows three digits in total with two after the point. A value of 3.85 fits. A value of 12.50 does not.

CONFIRM THE TABLE EXISTS

Open the Object Browser and click TEST_STUDENTS. The Columns tab lists your six columns and their types. The Data tab is empty, which is expected at this point. This is the Object Browser doing its job: you checked your work without writing a query.

Part 4 | Add Data

Start with one record so you can see the shape of an INSERT.

```
SQL
INSERT INTO test_students
VALUES (1, 'Alpha', 'Tester01', 'Computer Science', 3.85, DATE '2024-08-26');
```

You should see the message "1 row(s) inserted." Three details matter here. Text goes in single quotes. Numbers do not. A date is written as the word DATE followed by a quoted value in YYYY-MM-DD form.

The values must appear in the same order as the columns in your CREATE TABLE statement.

ADD THE OTHER NINE RECORDS

SQL Commands runs only one statement per Run, so nine more INSERT statements would mean nine trips. INSERT ALL avoids that. It is a single statement that adds many records at once. Paste the whole block and click Run.

```
SQL
INSERT ALL
  INTO test_students VALUES (2, 'Bravo',  'Tester02','Information Systems',3.42,DATE '2024-08-26')
  INTO test_students VALUES (3, 'Charlie','Tester03','Computer Science',   3.10,DATE '2025-01-13')
  INTO test_students VALUES (4, 'Delta',  'Tester04','Data Science',       3.95,DATE '2024-08-26')
  INTO test_students VALUES (5, 'Echo',   'Tester05','Information Systems',2.87,DATE '2025-01-13')
  INTO test_students VALUES (6, 'Foxtrot','Tester06','Computer Science',   3.66,DATE '2025-01-13')
  INTO test_students VALUES (7, 'Golf',   'Tester07','Data Science',       3.21,DATE '2025-08-25')
  INTO test_students VALUES (8, 'Hotel',  'Tester08', NULL,                2.55,DATE '2025-08-25')
  INTO test_students VALUES (9, 'India',  'Tester09','Information Systems',3.78,DATE '2025-08-25')
  INTO test_students VALUES (10,'Juliet', 'Tester10','Data Science',       3.33,DATE '2024-08-26')
SELECT * FROM dual;
```

The closing line SELECT * FROM dual; is required. DUAL is a built-in table with one row. Oracle uses it when a statement needs something to select from but the data is already supplied. You should see the message "9 row(s) created."

> **Note the NULL:** Student 8 has no major on record. That is deliberate. NULL means no value is present. It is not the same as an empty string or a zero, and you will query for it in Part 5.

Check the count:

```
SQL
SELECT COUNT(*) FROM test_students;
```

You should get 10. A result of 1 means the INSERT ALL block did not run. A result above 10 means you ran it twice. In that case, run DELETE FROM test_students; and start Part 4 again.

Part 5 | Query the Data

SELECT reads records back out of a table. The asterisk means every column.

```sql
SELECT * FROM test_students;
```

To return only some columns, list them instead of the asterisk:

```sql
SELECT first_name, last_name, gpa FROM test_students;
```

You can rename a column in the output without changing the table, and DISTINCT removes duplicate values:

```sql
SELECT DISTINCT major AS "Major" FROM test_students;
```

> **Row order:** Rows may come back in a different order than you inserted them. A table has no built-in order. When order matters, ask for it with ORDER BY.

FILTER ROWS WITH WHERE

A query without a WHERE clause returns every row. WHERE narrows the result down, and it is the piece of SQL you will write most often.

```sql
SELECT * FROM test_students WHERE major = 'Computer Science';
```

Text comparisons are case sensitive. A search for 'computer science' returns nothing. The usual operators all work: = for equal, <> for not equal, and > < >= <= for ranges.

```sql
SELECT * FROM test_students WHERE gpa > 3.5;
```

COMBINE CONDITIONS

```sql
SELECT first_name, last_name, gpa
FROM test_students
WHERE major = 'Data Science' AND gpa > 3.3;
```

AND requires both conditions. OR requires either one. When you mix them, add parentheses so the grouping is clear.

LIKE, IN, AND BETWEEN

```sql
SELECT * FROM test_students WHERE first_name LIKE 'C%';
```

The percent sign stands for any number of characters, so this finds first names that begin with C. An underscore stands for exactly one character, so '_olf' matches Golf.

```sql
SELECT first_name, major
FROM test_students
WHERE major IN ('Computer Science', 'Data Science');
```

IN is shorter and clearer than several OR conditions in a row. BETWEEN checks a range, and it includes both endpoints:

```sql
SELECT * FROM test_students WHERE gpa BETWEEN 3.0 AND 3.5;
```

FIND MISSING VALUES WITH IS NULL

```sql
SELECT * FROM test_students WHERE major IS NULL;
```

> **Watch out:** The condition major = NULL always returns nothing, even for a row with no major. NULL is not a value you can equal. Use IS NULL and IS NOT NULL instead.

SORT WITH ORDER BY

```sql
SELECT first_name, last_name, gpa FROM test_students ORDER BY gpa DESC;
```

DESC sorts from high to low. ASC sorts from low to high, and ASC is the default. Sorting by more than one column breaks ties. The clause order matters: SELECT, then FROM, then WHERE, then ORDER BY.

```sql
SELECT first_name, last_name, gpa
FROM test_students
WHERE major = 'Computer Science'
ORDER BY gpa DESC;
```

Part 6 | Summarize the Data

Every query so far returns rows as they are stored. Aggregate functions do something different. They collapse many rows into a single answer.

Function	What it returns
COUNT(*)	The number of rows that match
AVG(column)	The average of a numeric column
MIN(column) and MAX(column)	The smallest and the largest value
SUM(column)	The total of a numeric column

```sql
SELECT COUNT(*), ROUND(AVG(gpa), 2), MIN(gpa), MAX(gpa)
FROM test_students;
```

GROUP THE RESULTS

GROUP BY runs the aggregate once per group instead of once for the whole table:

```sql
SELECT major, COUNT(*) AS "Students", ROUND(AVG(gpa), 2) AS "Average GPA"
FROM test_students
GROUP BY major
ORDER BY major;
```

You get one row per major, plus a row with an empty major for student 8. Every column in the SELECT list must either sit inside an aggregate function or appear in the GROUP BY clause. Leaving one out produces ORA-00979, the most common error in this part.

FILTER THE GROUPS WITH HAVING

```sql
SELECT major, COUNT(*) AS "Students"
FROM test_students
GROUP BY major
HAVING COUNT(*) > 2;
```

> **WHERE or HAVING?** WHERE filters individual rows before they are grouped. HAVING filters the groups afterwards. A condition that uses COUNT, AVG, or another aggregate belongs in HAVING.

Part 7 | Change Data and Structure

UPDATE

UPDATE changes values in records that already exist.

```sql
UPDATE test_students SET major = 'Data Science' WHERE student_id = 8;
```

Verify the change before you move on:

```sql
SELECT * FROM test_students WHERE student_id = 8;
```

Now set the value back, so the practice set in Part 9 still works as written. A column is set to NULL the same way it is set to any other value:

```sql
UPDATE test_students SET major = NULL WHERE student_id = 8;
```

> **Warning:** An UPDATE without a WHERE clause changes every row in the table. Write the WHERE clause first, then go back and fill in the SET clause.

DELETE

DELETE removes whole records.

```sql
DELETE FROM test_students WHERE student_id = 10;
```

Put the record back before you move on, so your data is complete:

```
SQL
INSERT INTO test_students
VALUES (10, 'Juliet', 'Tester10', 'Data Science', 3.33, DATE '2024-08-26');
```

> **Warning:** DELETE FROM test_students; with no WHERE clause removes every row and leaves an empty table behind. There is no undo once the change is committed.

ALTER TABLE

INSERT, SELECT, UPDATE, and DELETE all work on data. ALTER TABLE changes the table itself. Add a column:

```
SQL
ALTER TABLE test_students ADD (email VARCHAR2(60));
```

Existing rows get NULL in the new column. Fill one in, then widen a column, then drop the new column again:

```
SQL
UPDATE test_students SET email = 'tester01@example.com' WHERE student_id = 1;

SQL
ALTER TABLE test_students MODIFY (major VARCHAR2(60));

SQL
ALTER TABLE test_students DROP COLUMN email;
```

> **Warning:** Dropping a column deletes everything stored in it. Unlike DELETE, this changes the structure of the table and not only its contents.

The five commands so far

Command	What it does	Example
SELECT	Reads data	`SELECT * FROM test_students WHERE gpa > 3.5;`
INSERT	Adds data	`INSERT INTO test_students VALUES (11, 'Kilo', 'Tester11', 'Data Science', 3.40, DATE '2025-08-25');`
UPDATE	Changes data	`UPDATE test_students SET gpa = 3.90 WHERE student_id = 1;`
DELETE	Removes data	`DELETE FROM test_students WHERE student_id = 11;`
ALTER TABLE	Changes the table	`ALTER TABLE test_students ADD (email VARCHAR2(60));`

UPDATE and DELETE almost always need a WHERE clause. SELECT is the safe command, because it never changes anything. Run a SELECT with your WHERE clause first, and check that it returns the rows you meant to change.

Part 8 | Write and Run a Script

Everything so far went through SQL Commands, one statement at a time. That works while you are exploring. It stops working when you need to rebuild the same table tomorrow, or hand your setup to a teammate, or submit it with an assignment.

SQL Scripts solves that. A script is a named file of SQL statements stored in your workspace. You run the whole file with one click, and APEX reports on every statement in it.

Why scripts matter

SQL Commands	SQL Scripts
One statement per Run	Many statements in one run
Nothing is kept unless you save it	Saved under a name in your workspace
Good for testing an idea	Good for a job you repeat
Results appear in a grid	A run report lists every statement and its outcome
Hard to share	Can be downloaded, uploaded, and handed to someone else

Create your first script

Go to SQL Workshop, then SQL Scripts, then click Create. Give the script the name rebuild_test_students and type the following into the editor.

Each statement ends with a semicolon. The lines that start with two dashes are comments, and Oracle ignores them.

```
SCRIPT
-- rebuild_test_students
-- Rebuilds the demo table from nothing.
-- Remove the two dashes on the DROP line when you rerun this script.

-- DROP TABLE test_students;

CREATE TABLE test_students (
  student_id   NUMBER,
  first_name   VARCHAR2(30),
  last_name    VARCHAR2(30),
  major        VARCHAR2(40),
  gpa          NUMBER(3,2),
  enrolled_on  DATE
);
```

Below that, in the same script, add the data and commit it:

```
SCRIPT
INSERT ALL
  INTO test_students VALUES (1, 'Alpha',  'Tester01','Computer Science',   3.85,DATE '2024-08-26')
  INTO test_students VALUES (2, 'Bravo',  'Tester02','Information Systems',3.42,DATE '2024-08-26')
  INTO test_students VALUES (3, 'Charlie','Tester03','Computer Science',   3.10,DATE '2025-01-13')
  INTO test_students VALUES (4, 'Delta',  'Tester04','Data Science',       3.95,DATE '2024-08-26')
  INTO test_students VALUES (5, 'Echo',   'Tester05','Information Systems',2.87,DATE '2025-01-13')
  INTO test_students VALUES (6, 'Foxtrot','Tester06','Computer Science',   3.66,DATE '2025-01-13')
  INTO test_students VALUES (7, 'Golf',   'Tester07','Data Science',       3.21,DATE '2025-08-25')
  INTO test_students VALUES (8, 'Hotel',  'Tester08', NULL,                2.55,DATE '2025-08-25')
  INTO test_students VALUES (9, 'India',  'Tester09','Information Systems',3.78,DATE '2025-08-25')
  INTO test_students VALUES (10,'Juliet', 'Tester10','Data Science',       3.33,DATE '2024-08-26')
SELECT * FROM dual;

COMMIT;
```

RUN IT AND READ THE REPORT

1. Click Save to keep the script in your workspace.
2. Click Run, then Run Now on the confirmation page.
3. APEX opens the run report. It shows each statement, the rows it affected, and any error it raised.
4. Check the summary line at the top. It should say the statements were processed with zero errors.
5. Open the Object Browser and confirm TEST_STUDENTS holds ten records again.

Why COMMIT is in the script: SQL Commands normally commits for you. A script does not always do the same. Ending a data-loading script with COMMIT; makes the result the same every time, on any workspace.

About the commented DROP line: The first time you run this script the table does not exist, so DROP would fail. Leave the line commented for the first run. Remove the two dashes when you want the script to wipe the old table and rebuild from scratch.

A second script: a summary report

Scripts are not only for setup. A script can hold several queries that you want to see together. Create a second script named test_students_report with these three statements.

```
SCRIPT
-- test_students_report
-- Three views of the demo data in one run.

SELECT COUNT(*) AS "Total students" FROM test_students;

SELECT major, COUNT(*) AS "Students", ROUND(AVG(gpa), 2) AS "Average GPA"
FROM test_students
GROUP BY major
ORDER BY major;

SELECT first_name, last_name, gpa
FROM test_students
ORDER BY gpa DESC
FETCH FIRST 3 ROWS ONLY;
```

Run it. The report shows the output of all three queries on one page, in the order they appear in the script. The last statement uses FETCH FIRST, which limits a result to a set number of rows.

WHAT ELSE SQL SCRIPTS GIVES YOU

- Every script you save stays in the workspace, so you can run it again in a later session.
- The Download button saves a script as a .sql file you can keep with your assignment work.
- The Upload button loads a .sql file that someone else wrote, including a file from your instructor.
- The Results tab keeps a history of past runs, so you can compare one run against another.

> **Which tool, when:** Use SQL Commands while you work out what a statement should say. Move the finished statements into a script once you know they are right. That is how database work is done outside a classroom.

Part 9 | Practice

Nine short exercises. Write each answer yourself before you check the solutions. If a result looks wrong, run SELECT * FROM test_students; and confirm your data still matches what Part 4 built.

Set A | Queries

1. Show the first name, last name, and major for every student.
2. Show all students with a GPA above 3.5.
3. Show students whose first name begins with the letter F.
4. Show the student with no major on record.
5. List every student sorted by GPA, from highest to lowest.

Set B | Summaries

1. Show the average GPA for each major, rounded to two decimal places.
2. Show only the majors with more than two students.

Set C | Scripts

1. Create a script named add_student. It should add student 11 (Kilo, Tester11, Data Science, GPA 3.40, enrolled 25 August 2025), commit the change, and then show the row count. Save it and run it.
2. Add two more statements to the same script: one that deletes student 11, and one that shows the row count again. Run the script and check the report says every statement finished with no errors.

> **Before any UPDATE or DELETE:** Write the SELECT version of your WHERE clause first and run it. If it returns the rows you meant to change, swap SELECT for UPDATE or DELETE. This habit will save you at least once this semester.

Solutions

SET A

```
A1
SELECT first_name, last_name, major FROM test_students;

A2
SELECT * FROM test_students WHERE gpa > 3.5;

A3
SELECT * FROM test_students WHERE first_name LIKE 'F%';

A4
SELECT * FROM test_students WHERE major IS NULL;

A5
SELECT * FROM test_students ORDER BY gpa DESC;
```

SET B

```
B1
SELECT major, ROUND(AVG(gpa), 2) AS "Average GPA"
FROM test_students
GROUP BY major;

B2
SELECT major, COUNT(*) AS "Students"
FROM test_students
GROUP BY major
HAVING COUNT(*) > 2;
```

SET C

```
C1
-- add_student
INSERT INTO test_students
VALUES (11, 'Kilo', 'Tester11', 'Data Science', 3.40, DATE '2025-08-25');

COMMIT;

SELECT COUNT(*) FROM test_students;

C2  (ADDED BELOW THE STATEMENTS FROM C1)
-- add_student, extended
DELETE FROM test_students WHERE student_id = 11;

COMMIT;

SELECT COUNT(*) FROM test_students;
```

> **If your answer differs:** More than one correct query often exists. The result set is what matters, not the exact wording. Compare what came back, not the text you typed.

Part 10 | Troubleshooting

What you see	What it means	What to do
ORA-00955: name is already used by an existing object	A table with that name already exists in your schema.	Run DROP TABLE test_students; and create it again, or pick a different name.
ORA-00942: table or view does not exist	Oracle cannot find the table.	Check the spelling, then check that the Schema selector is set to your schema.
An error when you paste several statements at once	SQL Commands runs one statement per Run.	Run each statement on its own, or move them into a SQL script.
ORA-00979: not a GROUP BY expression	A column in your SELECT list is neither aggregated nor grouped.	Add that column to the GROUP BY clause, or wrap it in an aggregate function.
ORA-12899: value too large for column	Your text is longer than the column allows.	Shorten the value, or widen the column with ALTER TABLE MODIFY.
ORA-01722: invalid number	A quoted value was used where a number belongs.	Remove the quotes. Write gpa = 3.5, not gpa = '3.5'.
ORA-01861: literal does not match format string	A date was written in a format Oracle did not expect.	Use DATE '2025-01-13', with the four digit year first.
A query for NULL returns nothing	The condition used = NULL.	Use IS NULL or IS NOT NULL instead.
Only 10 rows appear	The Rows dropdown defaults to 10.	Raise the Rows value and run the query again.
A script reports errors on some statements	The run report flags each failed statement.	Open the report, read the error next to that statement, fix that line, and run the script again.
Your records are gone in a later session	The changes were never committed.	Run COMMIT; after your inserts, and end every data-loading script with COMMIT;.

START OVER

If your data reaches a state you cannot untangle, run the rebuild script from Part 8 with the DROP line uncommented. That is exactly the problem a saved script is meant to solve.

QUESTIONS

Contact your instructor first. For help with the steps in this document, you can also contact the teaching assistant. Both addresses are on page 1.